

RoK4 Reward Structure

작성일: 2026-07-15

이 문서는 RoK4-Isaac-Velocity-Flat-v0 task의 현재 reward 구조와 reward function 설정을 정리한다. 현재 reward는 Isaac Lab G1 velocity task 구조를 RoK4 링크/관절 이름에 맞게 이식한 flat walking 초기 baseline이다. 최종 튜닝값이 아니라 첫 보행 실험을 위한 시작점으로 봐야 한다.

관련 파일

역할	파일
RoK4 flat task 설정	/home/rclab/rok4_lab/source/rok4_tasks/rok4_tasks/manager_based/locomotion/velocity/config/rok4/flat_env_cfg.py
RoK4 domain randomization 설정	/home/rclab/rok4_lab/source/rok4_tasks/rok4_tasks/manager_based/locomotion/velocity/config/rok4/domain_randomization_cfg.py
공통 locomotion reward term 기본값	/home/rclab/IsaacLab/source/isaaclab_tasks/isaaclab_tasks/manager_based/locomotion/velocity/velocity_env_cfg.py
locomotion 전용 reward 함수	/home/rclab/IsaacLab/source/isaaclab_tasks/isaaclab_tasks/manager_based/locomotion/velocity/mdp/rewards.py
Isaac Lab 공통 reward 함수	/home/rclab/IsaacLab/source/isaaclab/isaaclab/envs/mdp/rewards.py
mdp namespace 연결	/home/rclab/IsaacLab/source/isaaclab_tasks/isaaclab_tasks/manager_based/locomotion/velocity/mdp/__init__.py

구조 요약

RoK4FlatEnvCfg 는 환경 설정이고, RoK4RewardsCfg 는 reward term 묶음이다. Reward 구조는 두 가지 관계로 나누어 보면 될 것같린다.

첫 번째는 config class의 상속/포함 관계이다.

```
RoK4FlatEnvCfg
├─ 상속: LocomotionVelocityRoughEnvCfg
│   └─ 위치:
│       IsaacLab/.../locomotion/velocity/velocity_env_cfg.py
├─ rewards: RoK4RewardsCfg()
│   └─ RoK4RewardsCfg
│       ├── 상속: RewardsCfg
│       │   └─ 위치:
│       │       IsaacLab/.../locomotion/velocity/velocity_env_cfg.py
│       ├── 부모 RewardsCfg에서 물려받은 reward terms
│       │   ├── lin_vel_z_l2
│       │   ├── ang_vel_xy_l2
│       │   ├── flat_orientation_l2
│       │   └── undesired_contacts
│       └─ RoK4RewardsCfg에서 새로 정의/override한 reward terms
│           ├── termination_penalty
│           ├── track_lin_vel_xy_exp
│           ├── track_ang_vel_z_exp
│           ├── feet_air_time
│           ├── feet_slide
│           ├── dof_pos_limits
│           ├── action_pos_limits
│           ├── joint_deviation_hip
│           ├── joint_deviation_torso
│           ├── dof_acc_l2
│           ├── dof_torques_l2
│           ├── joint_vel_l2
│           ├── action_rate_l2
│           └── second_action_rate_l2
```

두 번째는 각 reward term이 실제 계산 함수를 참조하는 관계이다.

```

RoK4FlatEnvCfg
├── rewards = RoK4RewardsCfg()
│   └── RoK4RewardsCfg extends RewardsCfg
│       └── 각 reward term은 RewardTermCfg/RewTerm
│           └── func=mdp.xxx
│               ├── RoK4 로컬 mdp 함수
│               │   └── source/rok4_tasks/.../velocity/mdp/rewards.py
│               ├── Isaac Lab 공통 mdp 함수
│               │   └── IsaacLab/source/isaacsim/isaacsim/envs/mdp/rewards.py
│               └── locomotion velocity 전용 mdp 함수
│                   └── IsaacLab/source/isaacsim_tasks/.../velocity/mdp/rewards.py

```

즉 RoK4FlatEnvCfg -> RoK4RewardsCfg 는 상속이 아니라 포함/사용 관계이고, RoK4RewardsCfg -> RewardsCfg 는 상속 관계이다. mdp/rewards.py 는 부모 클래스가 아니라 실제 reward 계산 함수가 들어 있는 함수 모음이다. RoK4는 로컬 rok4_tasks...velocity.mdp 를 import하며, 이 로컬 mdp는 Isaac Lab의 기존 locomotion mdp 를 다시 export하고 RoK4 전용 reward 함수만 추가한다.

실제로는 다음처럼 이해하면 된다.

```

RoK4RewardsCfg는 RewardsCfg를 상속받아서 기본 reward term들을 물려받는다.
RoK4에 필요한 reward는 mdp.xxx 함수를 직접 지정해 새로 정의하거나 override한다.
RewardTermCfg/RewTerm은 mdp.xxx 함수, weight, params를 하나의 reward term으로 묶는다.

```

파일 배치 기준은 다음과 같다.

```

flat_env_cfg.py
├── 어떤 reward term을 쓸지, weight를 얼마로 둘지, 어떤 body/joint에 적용할지 결정

velocity/mdp/rewards.py
├── reward raw value를 어떻게 계산할지 정의

```

따라서 action_pos_limits, dof_torques_l2, joint_vel_l2, dof_acc_l2, action_rate_l2, second_action_rate_l2 처럼 RoK4 전용 weighting을 쓰는 계산식은 velocity/mdp/rewards.py 에 두고, weight를 얼마로 사용할지는 RoK4RewardsCfg 에 둔다. 이 term은 이전 action 두 개가 필요하므로 reset 직후 첫 두 policy step에서는 penalty를 0으로 둔다.

Action-rate reward의 action 기준

현재 RoK4Lab의 action_rate_l2 와 second_action_rate_l2 는 env.action_manager.action 을 기준으로 현재 action과 history의 차분을 만든다. 이 action buffer 자체는 scale 적용 전 raw policy action이지만, reward 함수는 차분에 ROK4_ACTION_SCALE 을 곱한 뒤 제곱합을 계산한다.

다만 RoK4FlatPP0RunnerCfg 에서 clip_actions = 1.0 을 명시하므로 Isaac Lab train.py 와 play.py 를 통해 실행할 때는 RslrlVecEnvWrapper 가 action을 먼저 [-1, 1] 로 clamp한다. 따라서 reward가 보는 action은 wrapper에서 [-1, 1] 로 잘린 raw action이며, penalty가 측정하는 값은 scaled joint-target offset 차분이다.

```

actor output
-> clip_actions = 1.0
-> clipped_raw_action
├── last_action observation = clipped_raw_action
├── scaled_action = clipped_raw_action * ROK4_ACTION_SCALE
│   ├── action_rate_l2 / second_action_rate_l2
│   └── q_target = default_joint_pos + scaled_action
│       └── action_pos_limits compares q_target with soft_joint_pos_limits

```

기존 IsaacGym RoK4 코드에서는 actuator_actions_raw * action_scale 으로 만든 actuator_actions 를 penalty_action_rate 와 penalty_action_2nd_rate 에 넘겼다. 즉 Gym 버전은 scaled actuator action 기준이고, 현재 RoK4Lab도 동일하게 scaled action 차분을 기준으로 한다. 또한 ROK4_ACTION_SCALE 값도 Gym의 [0.4, 0.5, 1.25, 1.5, 0.75, 0.75] 좌우 다리 scale과 torso 0.4 에 맞췄다. 다만 observation의 last_action 은 Isaac Lab convention에 따라 clipped raw action으로 유지한다.

Action target의 default_joint_pos 도 Gym의 활성 gait-ready pose와 동일하다. Hip pitch는 -0.0924 rad, knee pitch는 0.345 rad, ankle pitch는 -0.253 rad 이며 나머지 관절은 0.0 rad 이다. 따라서 joint_deviation_* reward의 기준 자세와 use_default_offset=True 가 사용하는 action 중심도 같은 Gym gait-ready 자세를 따른다.

Reward와 Domain Randomization의 구분

RoK4RewardsCfg 와 부모 RewardsCfg 는 reward term을 정의한다. 반면 foot friction, restitution, base/upper/lower body mass, base/upper/lower COM, reset pose 같은 domain randomization 값은 domain_randomization_cfg.py 에서 관리한다.

두 설정은 학습 결과에 모두 영향을 주지만 역할은 다르다.

구분	파일	역할
Reward	flat_env_cfg.py 의 RoK4RewardsCfg 및 self.rewards.xxx	policy가 어떤 행동을 더 좋게 볼지 점수 함수를 정의

구분	파일	역할
DR	domain_randomization_cfg.py	물성, 질량, COM, 초기상태 perturbation을 바꿔 다양한 상황에서 버티게 함

예를 들어 발 미끄러짐은 feet_slide reward가 penalty로 줄이고, foot-ground 마찰 계수는 domain_randomization_cfg.py 의 ROK4_STATIC_FRICTION_RANGE 와 ROK4_DYNAMIC_FRICTION_RANGE 가 정한다. 따라서 미끄러짐 문제를 볼 때는 reward와 DR을 함께 확인해야 하지만, 코드상 관리 위치는 분리되어 있다.

Reward와 PPO entropy/KL의 구분

Reward term은 환경의 RewardManager 가 행동의 점수를 계산하는 항목이다. 반면 PPO의 entropy와 KL은 rsl_rl optimizer가 policy distribution을 업데이트할 때 사용하는 학습 지표다. 따라서 entropy_coef 나 desired_kl 은 RoK4RewardsCfg 의 reward weight가 아니다.

Reward와 PPO 지표 비교		
항목	현재 값/위치	역할
reward weight	flat_env_cfg.py 의 RewTerm(weight=...)	환경 행동의 positive reward 또는 penalty 크기를 결정
entropy_coef	rsl_rl_ppo_cfg.py 의 0.002	Gaussian policy의 exploration entropy를 유지하려는 optimizer 압력을 결정
desired_kl	rsl_rl_ppo_cfg.py 의 0.01	old/new policy 차이를 기준으로 adaptive learning rate를 조절

RoK4 로컬 scripts/rsl_rl/rok4_ppo.py 는 adaptive schedule에서 이미 계산되는 mini-batch KL을 모아 TensorBoard의 Loss/kl`` (iteration 평균)과 ``Loss/kl_max`` (iteration 최대)로 기록한다. 현재 5 learning epochs와 4 mini-batches를 사용하므로 iteration당 20개 KL 값을 집계한다. 이 로깅을 위해 Isaac Lab 또는 설치된 ``rsl_rl 파일을 수정하지 않는다.

Reward 계산 방식

Isaac Lab의 RewardManager 는 각 reward term의 raw value를 계산한 뒤 weight 를 곱하고, 모든 term을 합산한다.

```
total_reward =
    sum(weight_i * reward_function_i(env, params_i))
```

따라서 positive weight는 행동을 유도하고, negative weight는 penalty로 작동한다.

RoK4 전용 Reward Terms

아래 term들은 RoK4RewardsCfg 에서 직접 정의하거나 부모 RewardsCfg 의 term을 RoK4용으로 override한 항목이다.

term	function	weight	params	의미
termination_penalty	mdp.is_terminated	-200.0	없음	episode가 termination되면 큰 penalty를 준다.
track_lin_vel_xy_exp	mdp.track_lin_vel_xy_yaw_frame_exp	1.0	command_name="base_velocity", std=0.5	yaw-aligned robot frame에서 x/y 선 속도 command를 추종하게 한다.
track_ang_vel_z_exp	mdp.track_ang_vel_z_world_exp	1.0	command_name="base_velocity", std=0.5	world frame yaw angular velocity command를 추종하게 한다.
feet_air_time	mdp.feet_air_time_positive_biped	0.75	feet: L_Foot_Link, R_Foot_Link; threshold=0.55	양발 보행에서 한 발씩 공중에 있는 step pattern을 유도한다. Threshold는 정확한 주기 목표가 아니라 single-stance reward의 시간 상한이다.
feet_slide	mdp.feet_slide	-0.2	feet: L_Foot_Link, R_Foot_Link	지면 접촉 중인 발이 미끄러지는 것을 줄인다.
dof_pos_limits	mdp.joint_pos_limits	-1.0	ROK4_JOINT_ORDER 에 포함된 전체 13관절, preserve_order=True	각 관절이 USD hard limit의 95%로 계산된 soft joint limit을 넘은 양을 합산해 억제한다.
action_pos_limits	mdp.action_pos_limits	-0.001	joint_pos action target과 ROK4_JOINT_ORDER 의 전체 13관절	Gym의 penalty_action_limits 에 대응한다. processed_actions 로 계산된 목표 관절 위치가 95% soft limit을 넘은 양을 합산한다.

term	function	weight	params	의미
joint_deviation_hip	mdp.joint_deviation_l1	-0.1	.*_Hip_Yaw_Joint, .*_Hip_Roll_Joint	hip yaw/roll이 default pose에서 과도하게 벗어나지 않게 한다.
joint_deviation_torso	mdp.joint_deviation_l1	-1.0	Torso_Yaw_Joint	torso yaw가 default pose에서 과도하게 벗어나지 않게 한다.
dof_acc_l2	mdp.joint_acc_l2	-1.0e-8	전체 RoK4 joint; index 2,3,8,9 weight 0.5	Isaac Lab의 asset.data.joint_acc를 그대로 쓰되, Gym의 jointAcc=-0.0001을 acceleration 기준으로 환산해 약하게 적용한다.
dof_torques_l2	mdp.joint_torques_l2	-1.0e-5	전체 RoK4 joint; index 2,3,8,9 weight 0.5	explicit actuator가 PhysX에 전달한 asset.data.applied_torque를 사용하여 Gym penalty_torque와 같은 weighted squared sum을 계산한다.
joint_vel_l2	mdp.joint_vel_l2	-1.0e-4	전체 RoK4 joint; index 2,3,8,9 weight 0.5	asset.data.joint_vel의 weighted squared sum으로 Gym penalty_joint_vel에 대응한다.
action_rate_l2	mdp.action_rate_l2	-0.1	action 전체; index 2,3,8,9 weight 0.5	clipped_raw_action * ROK4_ACTION_SCALE의 1차 차분을 줄이되 hip pitch/knee action penalty는 완화한다.
second_action_rate_l2	mdp.second_action_rate_l2	-0.05	action 전체; index 2,3,8,9 weight 0.5	scaled action의 2차 차분, 즉 $s(a_t - 2a_{t-1} + a_{t-2})$ 를 줄여 action jerk를 완화한다. reset 직후 첫 두 policy step은 history가 부족하므로 0으로 처리한다.

Gym과 Lab의 action-limit 신호 차이

Gym 구현은 actuator action을 ADAPT 변환한 joint_actions 기준의 target을 검사했다. 현재 Lab action term은 articulation joint 13개를 직접 명령하므로 action_pos_limits는 실제 Lab PD에 전달되는 joint target을 검사한다. 따라서 limit penalty의 의미와 weight는 Gym에 맞지만, 과거 actuator-to-joint transmission 좌표계를 수치적으로 복제하는 항목은 아니다.

Gym과 Lab의 torque/velocity 신호 차이

기존 Gym 코드는 ADAPT transmission 변환 전의 actuator_torques와 actuator_vel을 penalty에 사용했다. 현재 Lab 모델은 별도의 actuator-space state를 노출하지 않으므로 explicit actuator가 PhysX joint에 실제로 전달한 asset.data.applied_torque와 articulation의 asset.data.joint_vel을 사용한다. 따라서 관절별 0.5 weighting과 reward weight는 Gym과 같지만, transmission 좌표계가 다르므로 raw penalty 값이 수치적으로 완전히 같지는 않다.

부모 RewardsCfg에서 상속받고 RoK4에서 수정한 Terms

아래 term들은 RewardsCfg에서 기본으로 제공되며, RoK4FlatEnvCfg.__post_init__()에서 weight나 body/joint 대상이 RoK4에 맞게 수정된다.

term	function	current weight	RoK4 설정	의미
lin_vel_z_l2	mdp.lin_vel_z_l2	-0.2	기본 robot root	base가 z 방향으로 튀는 움직임을 줄인다.
ang_vel_xy_l2	mdp.ang_vel_xy_l2	-0.05	기본 robot root	roll/pitch angular velocity를 줄인다.
flat_orientation_l2	mdp.flat_orientation_l2	-10.0	기본 robot root	몸이 기울어지는 것을 줄이고 upright 자세를 유도한다.
undesired_contacts	mdp.undesired_contacts	-1.0	Base_Link, Upper_Body_Link; threshold=1.0	발이 아닌 base/upper body 접촉을 penalty로 처리한다.

Reward Function 요약

function	위치	계산 의미
is_terminated	Isaac Lab 공통	termination manager의 terminated flag를 reward term으로 반환한다.

function	위치	계산 의미
track_lin_vel_xy_yaw_frame_exp	locomotion mdp	yaw frame x/y 선속도 command error에 $\exp(-error / \text{std}^2)$ 를 적용한다.
track_ang_vel_z_world_exp	locomotion mdp	world z축 angular velocity command error에 $\exp(-error / \text{std}^2)$ 를 적용한다.
feet_air_time_positive_biped	locomotion mdp	한 발 지지 상태의 air/contact time을 보상하며, command 크기가 작으면 0이 된다.
feet_slide	locomotion mdp	접촉 중인 foot의 horizontal linear velocity norm을 penalty로 계산한다.
lin_vel_z_l2	Isaac Lab 공통	base body-frame z velocity squared.
ang_vel_xy_l2	Isaac Lab 공통	base body-frame roll/pitch angular velocity squared sum.
flat_orientation_l2	Isaac Lab 공통	projected gravity의 x/y component squared sum으로 몸 기울기를 penalty화한다.
joint_torques_l2	RoK4 로컬 mdp	전체 joint의 applied torque weighted squared sum. Index 2,3,8,9 는 weight 0.5.
joint_vel_l2	RoK4 로컬 mdp	전체 joint velocity의 weighted squared sum. Index 2,3,8,9 는 weight 0.5.
joint_acc_l2	RoK4 로컬 mdp	asset.data.joint_acc 의 weighted squared sum. Index 2,3,8,9 는 weight 0.5.
action_rate_l2	RoK4 로컬 mdp	현재 action과 이전 action 차이의 weighted squared sum. Index 2,3,8,9 는 weight 0.5.
second_action_rate_l2	RoK4 로컬 mdp	현재 action, 이전 action, 이전-이전 action의 2차 차분 weighted squared sum. Index 2,3,8,9 는 weight 0.5.
joint_deviation_l1	Isaac Lab 공통	현재 joint position과 default joint position 차이의 absolute sum.
joint_pos_limits	Isaac Lab 공통	soft joint position limit을 넘은 정도를 합산한다.
action_pos_limits	RoK4 로컬 mdp	joint_pos action의 processed target이 soft joint position limit을 넘은 정도를 합산한다.
undesired_contacts	Isaac Lab 공통	지정 body의 contact force가 threshold를 넘은 contact 개수를 penalty로 반환한다.

Command와 Reward의 연결

현재 RoK4 flat command range는 다음과 같다.

```
lin_vel_x = (-0.1, 0.85)
lin_vel_y = (-0.3, 0.3)
ang_vel_z = (-0.6, 0.6)
```

이 command는 track_lin_vel_xy_exp, track_ang_vel_z_exp, feet_air_time 에 직접 영향을 준다. 특히 feet_air_time_positive_biped 는 x/y command norm이 0.1 이하이면 reward를 0으로 만든다. 즉 거의 정지 명령에서는 stepping reward가 강하게 작동하지 않는다.

UniformVelocityCommand 가 반환하는 [lin_vel_x, lin_vel_y, ang_vel_z] 는 robot base frame 기준 command다. 따라서 양의 lin_vel_x 는 로봇이 현재 바라보는 방향의 전진, 양의 lin_vel_y 는 로봇 기준 좌측 이동, 양의 ang_vel_z 는 z축 주위 반시계 방향 회전을 뜻한다. 선속도 tracking 함수는 실제 world 선속도에서 roll/pitch 를 제외한 yaw 회전만 역변환한 gravity-aligned yaw frame 속도와 x/y command를 비교한다. 반면 현재 각속도 tracking 함수는 command의 z 성분과 world-frame root_ang_vel_w[:, 2] 를 비교한다. 평평하고 직립한 상태에서는 두 z축이 거의 같지만, 몸체가 크게 기울면 완전히 동일한 표현은 아니다.

부모 command 설정의 heading_command=True 와 rel_heading_envs=1.0 도 그대로 적용된다. 따라서 학습 중 yaw command는 처음 표본화한 각속도를 그대로 유지하는 방식이 아니다. World frame target heading을 표본화하고, 매 step 0.5 * wrap_to_pi(target_heading - current_heading) 을 계산한 뒤 현재 ang_vel_z 범위인 (-0.6, 0.6) rad/s 로 clip한 결과를 base-frame command의 z 성분에 저장한다.

현재 feet_air_time 은 threshold=0.55 s, weight=0.75 를 사용한다. 따라서 최대 raw reward는 0.55 이고, 최대 pre-dt 항목 크기는 0.55 * 0.75 = 0.4125 이다. 이 설정은 긴 single stance를 기 존 threshold=0.4 보다 강하게 보상한다.

Contact sensor는 update_period=0.002 s 와 history_length=self.decimation=5 를 사용한다. 따라서 feet_slide, undesired_contacts, illegal_body_contact 처럼 net_forces_w_history 를 검사하는 항목은 최근 policy interval의 5개 physics contact sample을 확인한다. 반면 feet_air_time_positive_biped 는 이 force history가 아니라 sensor의 현재 current_air_time 과 current_contact_time 을 사용한다. 이 contact-force history는 policy observation history와 별개의 buffer이다.

Play 환경은 lin_vel_x=0.85 m/s 를 사용하고 lateral/yaw command는 0으로 고정한다. 별도의 Teleop 환경은 heading_command=False 와 긴 resampling interval을 사 용하고, gamepad 또는 keyboard의 [lin_vel_x, lin_vel_y, ang_vel_z] 를 동일한 base-frame command buffer에 직접 기록한다. 입력은 학습 범위 (-0.1, 0.85) m/s, (-0.3, 0.3) m/s, (-0.6, 0.6) rad/s 안으로 scale된다. 이는 inference command source만 바꾸며 reward 정의나 학습 checkpoint 자체를 변경하지 않는다.

Termination과 Reward의 연결

RoK4 flat task는 Isaac Lab의 TerminationsCfg 를 수정하지 않고 로컬 RoK4TerminationsCfg 로 상속한다. 부모의 time_out 은 유지하고 base_contact 는 비활성화한 뒤 illegal_body_contact 를 추가한다.

```
class RoK4TerminationsCfg(TerminationsCfg):
    base_contact = None
    illegal_body_contact = DoneTerm(
        func=mdp.illegal_contact,
        params={
            "sensor_cfg": SceneEntityCfg(
```

```
        "contact_forces",
        body_names=R0K4_ILLEGAL_CONTACT_BODY_NAMES,
    ),
    "threshold": 1.0,
},
)
```

R0K4_ILLEGAL_CONTACT_BODY_NAMES 에는 Base_Link, Upper_Body_Link 와 좌우 Hip_Yaw, Hip_Roll, Thigh, Calf, Ankle_Pitch, Ankle_Roll link가 포함된다. 정상 접촉 body인 L_Foot_Link 와 R_Foot_Link 만 제외된다. 선택된 body 중 하나라도 contact force가 1.0 N 을 넘으면 episode가 종료된다.

mdp.is_terminated 는 non-timeout termination을 감지하므로 illegal_body_contact 가 발생하면 termination_penalty=-200.0 가 함께 작동한다. 반면 부모에서 상속한 정상 time_out 에는 이 penalty가 적용되지 않는다. 여러 body가 동시에 접촉해도 termination flag는 boolean이므로 termination penalty는 한 번만 적용된다. Contact sensor는 상대 물체 종류를 구분하지 않은 net contact force를 사용하므로 self-collision으로 선택 body에 큰 접촉력이 생겨도 종료될 수 있다.

현재 설계 의도

현재 reward는 다음 목표를 동시에 가진다.

- 1. command velocity를 따라간다.
- 2. upright 자세를 유지한다.
- 3. 한 발씩 드는 biped stepping pattern을 만든다.
- 4. 발 미끄러짐을 줄인다.
- 5. torque, joint acceleration, action rate, second action rate를 줄여 움직임을 부드럽게 한다.
- 6. ankle limit, hip/torso deviation을 제한한다.
- 7. Foot를 제외한 body 접촉을 실패 종료로 처리한다.

튜닝 시 우선 확인할 항목

현상	먼저 볼 항목
너무 자주 넘어진다	termination_penalty, illegal_body_contact, self-collision, flat_orientation_l2, 초기 자세
거의 감지 않고 버틴다	track_lin_vel_xy_exp, command range, feet_air_time
발이 많이 미끄러진다	feet_slide, foot collision, friction, contact sensor
관절이 떨린다	action_rate_l2, second_action_rate_l2, dof_acc_l2, PD gain, action scale
토크가 과도하다	dof_torques_l2, effort limit, action scale
관절이 limit 근처로 간다	dof_pos_limits, action_pos_limits, 해당 joint의 default pose, action scale, USD hard limit